ATA 019 — API integrada, interface com login e testes de ponta a ponta

> **Data:** 16/10/2026
> **Projeto:** Plataforma local de gestão jurídica e automação assistida
> **Participantes:** Contratante/advogado; estagiário de Engenharia de Software (IA assistida)
> **Referência:** ATA 015 e ATA 018
> **Carga horária da semana (13/10 a 16/10, com feriado em 12/10):** 24 horas

Contexto

As regras de negócio (ATAs 016 a 018) existiam como código de serviço, sem acesso pela interface. Esta semana conecta tudo: o serviço HTTP local passa a expor uma API autenticada e o protótipo passa a usá-la.

Decisões

1. **Rotas JSON** para login, logout, painel, minutas (criar, editar, enviar, aprovar, devolver, nova versão, DOCX) e auditoria. O endpoint original de DOCX (ATA 013) foi mantido, agora exigindo login.
2. **O próprio serviço entrega o protótipo** em `http://127.0.0.1:8765/`. Assim a interface e a API ficam na mesma origem, sem depender de CORS aberto. Quando o HTML é aberto direto do disco, ainda funciona, e o CORS aceita apenas essa origem e o endereço local.
3. **Cabeçalhos de segurança:** `X-Content-Type-Options`, `Cache-Control: no-store` e política de conteúdo (CSP) restrita às próprias origens.
4. **Limites de entrada:** corpo da requisição de até 64 KB, JSON obrigatório, bloqueio de caminhos fora de `prototype/` (path traversal).
5. **Erros sem vazamento:** falhas inesperadas devolvem mensagem genérica; detalhes (que poderiam conter dados da minuta) não vão para a resposta nem para o log.
6. **Interface:** tela de login, painel com contadores reais, formulário que salva rascunho, lista de minutas com ações conforme o perfil, e visão de auditoria só para o advogado. Todo texto vindo da API é inserido como texto (nunca como HTML), prevenindo injeção de scripts (XSS).

Entrega realizada

- `services/document_service.py` reescrito (roteador, tratamento de erros, arquivos estáticos).
- `prototype/index.html`, novo `prototype/app.js` e ajustes em `prototype/styles.css`.
- `tests/test_api.py`: servidor real em porta livre, banco temporário e dados fictícios.

Validação técnica

Total acumulado: 31 testes aprovados. Os 6 novos verificam: rotas protegidas respondem 401 sem login; login inválido; fluxo completo (criar → aprovar fora de ordem 409 → enviar → estagiária aprovando 403 → advogado aprovando → edição bloqueada 409 → DOCX válido); entradas inválidas (400/404); entrega do protótipo e bloqueio de `..%2f`; painel e cadastros.

Também foi verificada visualmente a tela de login do protótipo no navegador.

Como executar:

```
python scripts\popular_dados_ficticios.py --db data\dev.db   # cria usuários fictícios
(senhas exibidas uma vez)
$env:ESTAGIO_DB = "data\dev.db"
python services\document_service.py                          # abrir http://127.0.0.1:8765/
```

Anexo técnico — trecho comentado

```
for (m, regex), funcao in ROTAS.items():             # tabela de rotas: (método, padrão) ->
função
    achou = regex.match(caminho)
    if m == metodo and achou:
        conexao = conectar(self.db_path)             # uma conexão por requisição
        try:
            raise funcao(self, conexao, *achou.groups())   # a função devolve uma Resposta
        finally:
            conexao.close()
```

Cada exceção de negócio é traduzida em um código HTTP: autenticação 401, permissão 403, não encontrado 404, estado inválido 409, validação 400.

Limitações e riscos

- Sem HTTPS (acesso restrito ao próprio computador). Antes de qualquer uso em rede, será necessário proxy com TLS.
- O token fica em `sessionStorage` do navegador; é apagado ao fechar a aba.

Pendências para o contratante

- Testar o fluxo no navegador com os usuários fictícios e informar ajustes de texto e layout.

Próximo passo

Cadastros de clientes, processos e prazos, com painel de prazos próximos.

Documento de acompanhamento do projeto. Não substitui documentos oficiais da UTFPR nem instrumentos jurídicos do escritório.